

NAME – PIYUSH MODANI CSAI-D ROLL 19 ML ASSIGNMENT 06

Use the diabetes data set from UCI and Pima Indians Diabetes data set for performing the following:

- Univariate analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis
- Bivariate analysis: Linear and logistic regression modeling
- Multiple Regression analysis
- Also compare the results of the above analysis for the two data sets.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew, kurtosis
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.datasets import load_diabetes
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from google.colab import files
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving diabetes.csv to diabetes (1).csv

```
pima = pd.read_csv("diabetes.csv")

print("Pima Dataset Shape:" , pima.shape)
print(pima.head())
```

```
Pima Dataset Shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0             6     148             72             35        0  33.6
1             1      85             66             29        0  26.6
2             8     183             64              0        0  23.3
```

```

3          1      89          66          23      94 28.1
4          0     137          40          35     168 43.1

```

```

DiabetesPedigreeFunction Age Outcome
0          0.627    50         1
1          0.351    31         0
2          0.672    32         1
3          0.167    21         0
4          2.288    33         1

```

```

diabetes = load_diabetes()
sklearn_df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
sklearn_df['target'] = diabetes.target

print("Sklearn Diabetes Shape:" , sklearn_df.shape)
print(sklearn_df.head())

```

Sklearn Diabetes Shape: (442, 11)

```

age      sex      bmi      bp      s1      s2      s3 \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142

s4      s5      s6 target
0 -0.002592  0.019907 -0.017646 151.0
1 -0.039493 -0.068332 -0.092204  75.0
2 -0.002592  0.002861 -0.025930 141.0
3  0.034309  0.022688 -0.009362 206.0
4 -0.002592 -0.031988 -0.046641 135.0

```

```
s6          0.207917
target      0.440563
binary_target 0.000000
dtype: float64
```

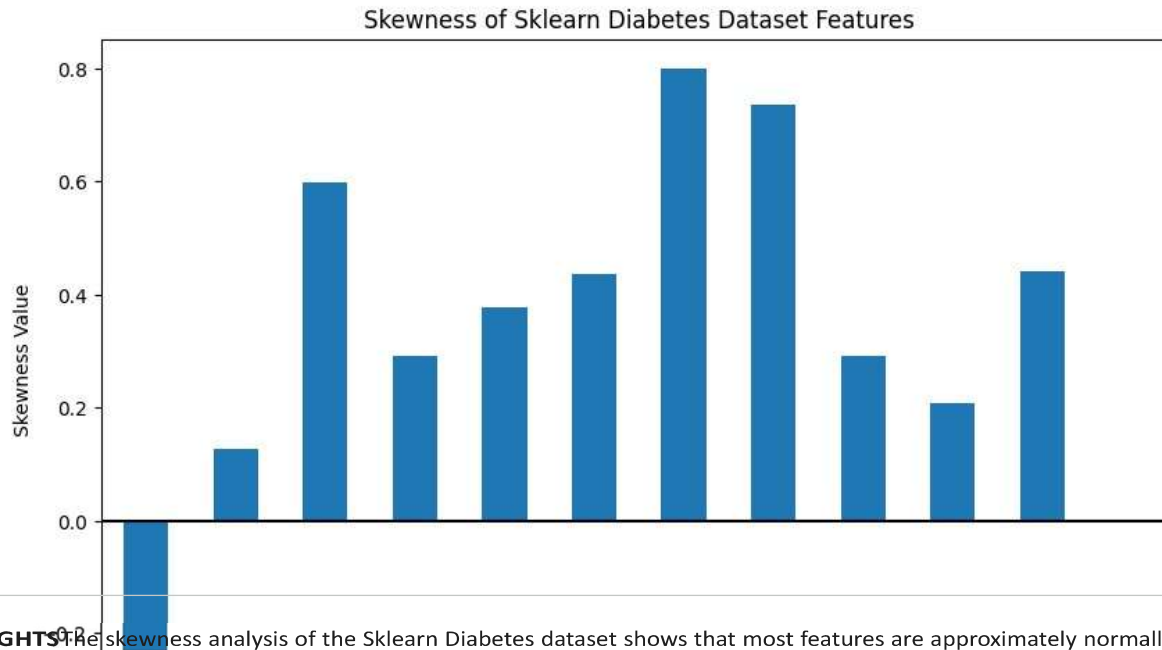
```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate skewness
skew_values = sklearn_df.skew()

print("Skewness of Each Feature:" )
print(skew_values)
```

```
Skewness of Each Feature:
age          -0.231382
sex           0.127385
bmi           0.598148
bp            0.290658
s1            0.378108
s2            0.436592
s3            0.799255
s4            0.735374
s5            0.291754
```

```
plt.figure(figsize=(10,6))
skew_values.plot(kind='bar')
plt.axhline(0, color='black')
plt.title("Skewness of Sklearn Diabetes Dataset Features")
plt.ylabel("Skewness Value")
plt.xticks(rotation=45)
plt.show()
```



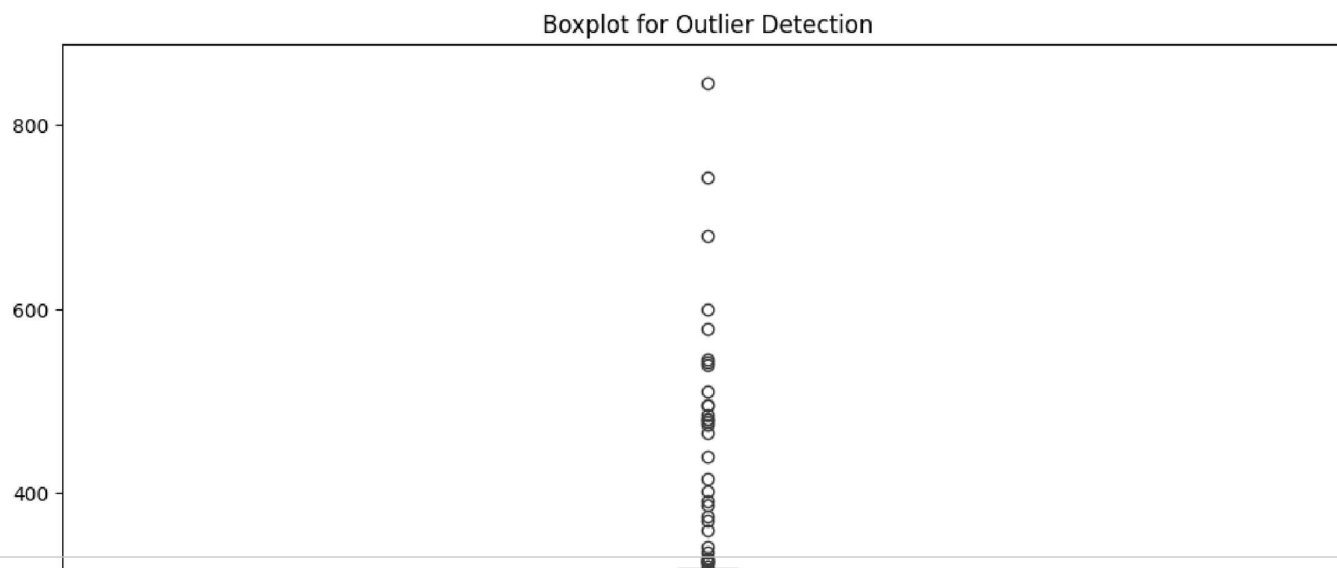
INSIGHTS The skewness analysis of the Sklearn Diabetes dataset shows that most features are approximately normally distributed, with minor positive and negative skewness. The target variable shows moderate right skewness, indicating a concentration of lower disease progression values with fewer extreme high values. No feature shows extreme skewness beyond 1, suggesting minimal transformation is required.

```
def univariate_analysis(df):
    result = pd.DataFrame({
        "Mean": df.mean(),
        "Median": df.median(),
        "Mode": df.mode().iloc[0],
        "Variance": df.var(),
        "Std Dev": df.std(),
```

```
        "Skewness": df.skew(),  
        "Kurtosis": df.kurt()  
    })  
    return result
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,8))
sns.boxplot(data=pima)
plt.title("Boxplot for Outlier Detection")
plt.xticks(rotation=90)
plt.show()
```



```
Q1 = pima.quantile( 0.25)
Q3 = pima.quantile( 0.75)
IQR = Q3 - Q1

pima_clean = pima[~((pima < (Q1 - 1.5 * IQR)) |
                    (pima > (Q3 + 1.5 * IQR))).any(axis=1)]

print("Original Shape:" , pima.shape)
print("After Removing Outliers:" , pima_clean.shape)
```

```
Original Shape: (768, 9)
After Removing Outliers: (639, 9)
```

```
def univariate_analysis(df):
    result = pd.DataFrame({
        "Mean": df.mean(),
        "Median": df.median(),
        "Mode": df.mode().iloc [0],
        "Variance": df.var(),
        "Std Dev": df.std(),
        "Skewness": df.skew(),
        "Kurtosis": df.kurt()
    })
    return result
```

```
print("Outliers in Pima Dataset" )
print(find_outliers(pima ))

print("\nOutliers in Sklearn Dataset" )
print(find_outliers(sklearn_df ))
```

Outliers in Pima Dataset

Pregnancies	4
Glucose	5
BloodPressure	45
SkinThickness	1
Insulin	34
BMI	19
DiabetesPedigreeFunction	29
Age	9
Outcome	0
dtype:	int64

Outliers in Sklearn Dataset

age	0
sex	0
bmi	3
bp	0

s1	8
s2	7
s3	7
s4	2
s5	4
s6	9
target	0
dtype:	int64


```
print("Univariate Analysis - Pima Dataset" )
print(univariate_analysis(pima ))

print("\nUnivariate Analysis - Sklearn Diabetes Dataset" )
print(univariate_analysis(sklearn_df ))
```

Univariate Analysis - Pima Dataset

	Mean	Median	Mode	Variance	\
Pregnancies	3.845052	3.0000	1.000	11.354056	
Glucose	120.894531	117.0000	99.000	1022.248314	
BloodPressure	69.105469	72.0000	70.000	374.647271	
SkinThickness	20.536458	23.0000	0.000	254.473245	
Insulin	79.799479	30.5000	0.000	13281.180078	
BMI	31.992578	32.0000	32.000	62.159984	
DiabetesPedigreeFunction	0.471876	0.3725	0.254	0.109779	
Age	33.240885	29.0000	22.000	138.303046	
Outcome	0.348958	0.0000	0.000	0.227483	

	Std Dev	Skewness	Kurtosis
Pregnancies	3.369578	0.901674	0.159220
Glucose	31.972618	0.173754	0.640780
BloodPressure	19.355807	-1.843608	5.180157
SkinThickness	15.952218	0.109372	-0.520072
Insulin	115.244002	2.272251	7.214260
BMI	7.884160	-0.428982	3.290443
DiabetesPedigreeFunction	0.331329	1.919911	5.594954
Age	11.760232	1.129597	0.643159
Outcome	0.476951	0.635017	-1.600930

Univariate Analysis - Sklearn Diabetes Dataset

	Mean	Median	Mode	Variance	Std Dev	Skewness	\
age	-1.444295e-18	0.005383	0.016281	0.002268	0.047619	-0.231382	
sex	2.543215e-18	-0.044642	-0.044642	0.002268	0.047619	0.127385	
bmi	-2.255925e-16	-0.007284	-0.030996	0.002268	0.047619	0.598148	
bp	-4.854086e-17	-0.005670	-0.040099	0.002268	0.047619	0.290658	
s1	-1.428596e-17	-0.004321	-0.037344	0.002268	0.047619	0.378108	
s2	3.898811e-17	-0.003819	-0.001001	0.002268	0.047619	0.436592	
s3	-6.028360e-18	-0.006584	-0.013948	0.002268	0.047619	0.799255	
s4	-1.788100e-17	-0.002592	-0.039493	0.002268	0.047619	0.735374	
s5	9.243486e-17	-0.001947	-0.018114	0.002268	0.047619	0.291754	
s6	1.351770e-17	-0.001078	0.003064	0.002268	0.047619	0.207917	
target	1.521335e+02	140.500000	72.000000	5943.331348	77.093005	0.440563	

	Kurtosis
age	-0.671224

```
sex      -1.992811
bmi       0.095094
bp       -0.532797
s1        0.232948
s2        0.601381
s3        0.981507
s4        0.444402
s5       -0.134367
s6        0.236917
target  -0.883057
```

```
print("Frequency of Outcome in Pima Dataset:" )
print(pima['Outcome'].value_counts())
```

```
Frequency of Outcome in Pima Dataset:
Outcome
0      500
1      268
Name: count, dtype: int64
```

```
import seaborn as sns
import matplotlib.pyplot as plt

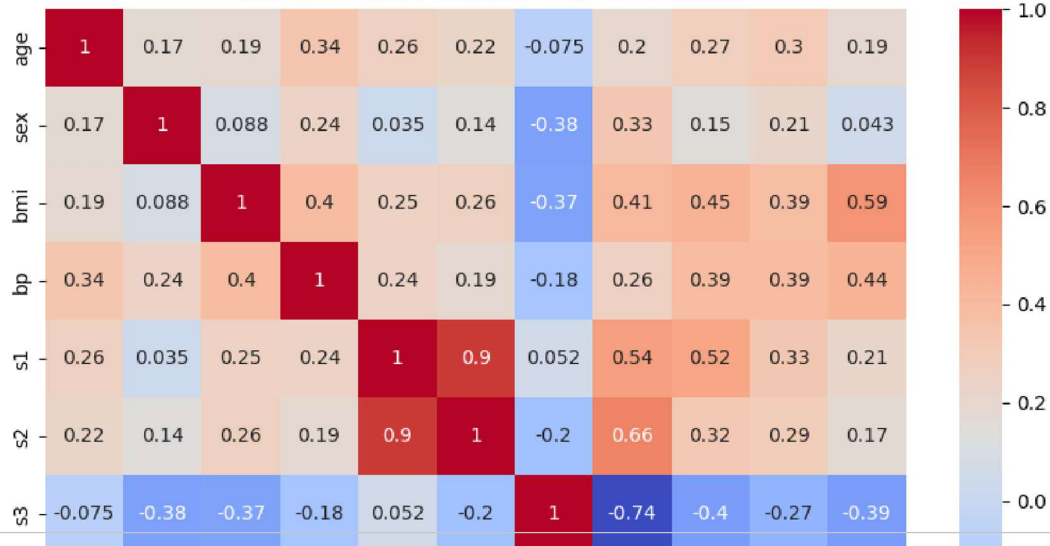
corr_matrix = sklearn_df.corr()

print(corr_matrix['target'].sort_values(ascending=False))
```

```
target    1.000000
bmi       0.586450
s5        0.565883
bp        0.441482
s4        0.430453
s6        0.382483
s1        0.212022
age       0.187889
s2        0.174054
sex       0.043062
s3       -0.394789
Name: target, dtype: float64
```

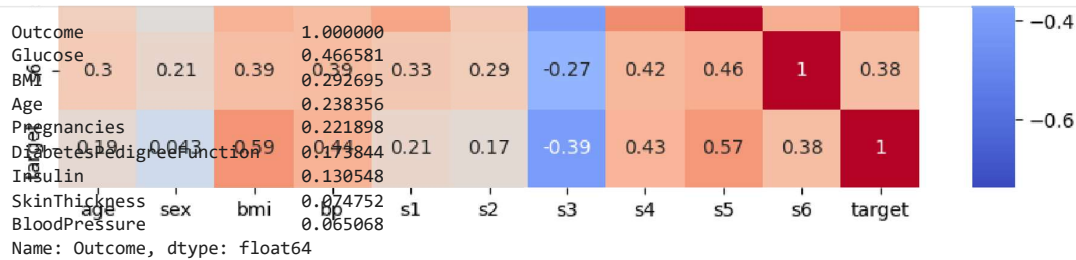
```
plt.figure(figsize=( 10,8))  
sns.heatmap(corr_matrix, annot= True, cmap='coolwarm')  
plt.title( "Correlation Matrix - Sklearn Diabetes Dataset" )  
plt.show()
```

Correlation Matrix - Sklearn Diabetes Dataset



```
corr_matrix_pima = pima.corr()
```

```
print(corr_matrix_pima['Outcome'].sort_values(ascending=False))
```



```
# Correlation with target
corr = sklearn_df.corr()[ 'target' ].sort_values(ascending =False)

print("Correlation with Target:" )
print(corr)
```

Correlation with Target:

target	1.000000
bmi	0.586450
s5	0.565883
bp	0.441482
s4	0.430453

s6	0.382483
s1	0.212022
age	0.187889
s2	0.174054
sex	0.043062
s3	-0.394789

Name: target, dtype: float64

```
best_feature = corr.index[ 1]  # First is target itself, second is highest correl ated feature
```

```
print("Best Single Feature:" , best_feature)
```

Best Single Feature: bmi

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.datasets import load_diabetes

# Load dataset
diabetes = load_diabetes()
sklearn_df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
sklearn_df['target'] = diabetes.target

# Select feature
X = sklearn_df[['s5']]
y = sklearn_df['target']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Model
model = LinearRegression()
model.fit(X_train, y_train)

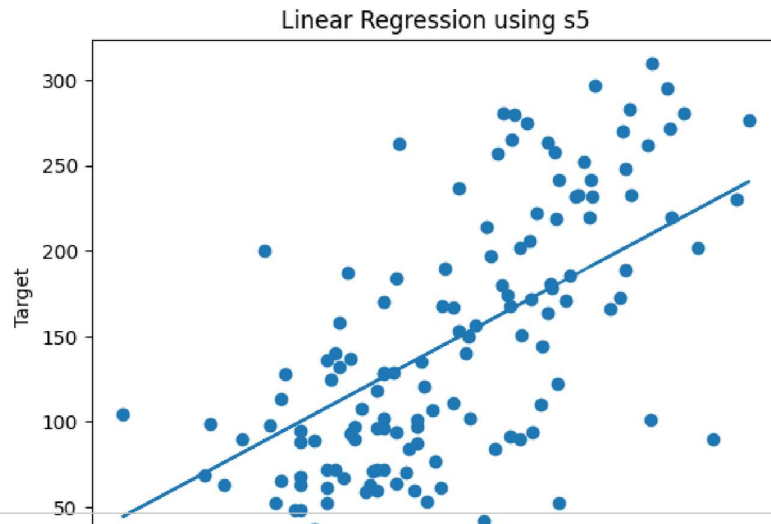
# Prediction
y_pred = model.predict(X_test)

print("Single Feature Linear Regression" )
print("Feature Used: s5" )
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))

# ----- GRAPH -----
plt.scatter(X_test, y_test)
```

```
plt.plot(X_test, y_pred)
plt.xlabel("s5")
plt.ylabel("Target")
plt.title("Linear Regression using s5")
plt.show()
```

Single Feature Linear Regression
Feature Used: s5
R2 Score: 0.43602358483676396
MSE: 3044.5182594962926



```
# Select top 3 correlated features (excluding target)
top_features = corr.index[ 1:4]

print("Top 3 Features for Multiple Regression:" , list(top_features))
```

Top 3 Features for Multiple Regression: ['bmi', 's5', 'bp']

```
X = sklearn_df[ list(top_features) ]
y = sklearn_df[ 'target' ]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size= 0.3, random_state= 42
)

multi_model = LinearRegression()
multi_model.fit(X_train, y_train)

y_pred = multi_model.predict(X_test)
```

```
print("\nMultiple Regression Results" )
print("Features Used:" , list(top_features))
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
Multiple Regression Results
Features Used: ['bmi', 's5', 'bp']
R2 Score: 0.47717443352284294
MSE: 2822.373313626039
```

```
corr_pima = pima.corr()[ 'Outcome' ].sort_values(ascending =False)
print("Correlation with Outcome:" )
print(corr_pima)
```

```
Correlation with Outcome:
Outcome          1.000000
Glucose          0.466581
BMI              0.292695
Age              0.238356
Pregnancies      0.221898
DiabetesPedigreeFunction 0.173844
Insulin          0.130548
SkinThickness    0.074752
BloodPressure    0.065068
Name: Outcome, dtype: float64
```



```
# =====  
# MULTIPLE LINEAR REGRESSION  
# CLEAN VERSION  
# =====  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import r2_score, mean_squared_error  
import matplotlib.pyplot as plt  
import numpy as np  
  
# IMPORTANT: Use original continuous target  
X = sklearn_df[['bmi', 's5', 'bp']]  
y = sklearn_df['target'] # NOT binary_target  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)  
  
model_multi = LinearRegression()  
model_multi.fit(X_train, y_train)
```

```
y_pred_multi = model_multi.predict(X_test)

print("R2:", r2_score(y_test, y_pred_multi))
print("MSE:", mean_squared_error(y_test, y_pred_multi))

# Convert to numpy (safe)
y_test_np = np.array(y_test)
y_pred_np = np.array(y_pred_multi)

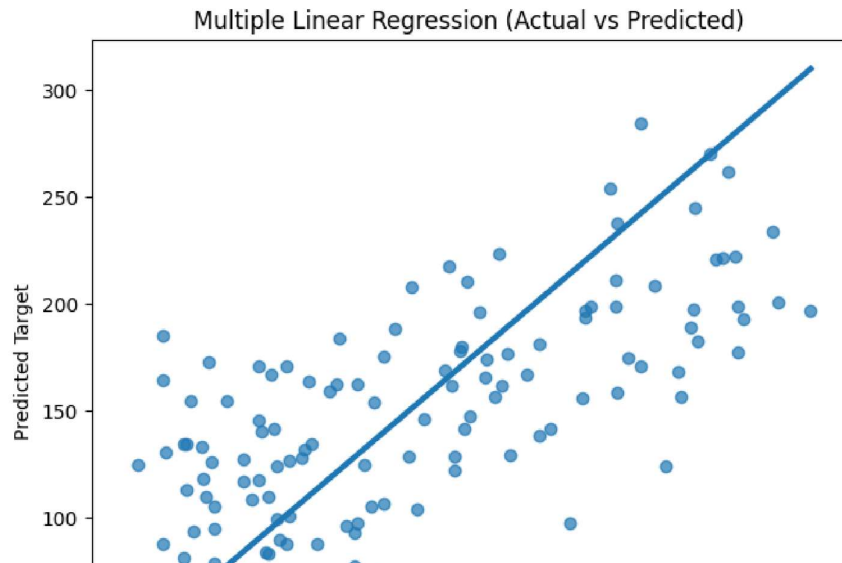
plt.figure(figsize=(7,6))
plt.scatter(y_test_np, y_pred_np, alpha=0.7)

min_val = min(y_test_np.min(), y_pred_np.min())
max_val = max(y_test_np.max(), y_pred_np.max())

plt.plot([min_val, max_val], [min_val, max_val], linewidth=3)

plt.xlabel("Actual Target")
plt.ylabel("Predicted Target")
plt.title("Multiple Linear Regression (Actual vs Predicted)")
plt.show()
```

R2: 0.47717443352284294
MSE: 2822.373313626039



```
# MULTIPLE LOGISTIC REGRESSION
# WITH PROPER SIGMOID CURVE
```



```
# 7 Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=( 6,5))
sns.heatmap(cm, annot= True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# 8 SUPER SMOOTH SIGMOID CURVE
#   (Vary BMI, Fix Others)

# Get coefficients
b0 = log_model.intercept_[ 0]
b1, b2, b3 = log_model.coef_[ 0]

# Create smooth BMI range
bmi_range = np.linspace(
    X_scaled[:, 0].min(),
    X_scaled[:, 0].max(),
    1000
)

# Since scaled, other features mean = 0
z = b0 + b1*bmi_range
y_sigmoid = 1 / (1 + np.exp(-z))

# Plot
plt.figure(figsize=( 8,6))

# Scatter original scaled BMI vs class
plt.scatter(X_scaled[:, 0], y, alpha=0.4)

# Plot sigmoid
plt.plot(bmi_range, y_sigmoid, linewidth= 4)

plt.xlabel("Scaled BMI")
plt.ylabel("Probability")
plt.title("Super Smooth Sigmoid Curve - Multiple Logistic Regression")
plt.ylim(-0.1,1.1)
plt.show()
```

Multiple Logistic Regression
Accuracy: 0.7819548872180451

